

Question Booklet

- The final exam contains 10 QUESTIONS and is scheduled for 4 HOURS.
- The maximum amount of points you can earn is 85.
- The minimum number of points to pass the exam is 40.
- Please verify if your question booklet consists of 12 PAGES (plus 3 pages with supplementary lecture slides) that are printed legibly, else contact the tutor immediately.
- The questions address both the theory part and the exercise part of the lecture. For questions marked with an asterisk, supplementary lecture slides are provided at the end of this booklet.
- No electronic devices (calculator, notebook, tablet, PDA, cell phone) are allowed.
- Answers without sufficient details are void (e.g.: you can't just say "yes" or "no" as the answer).
- Only the answer booklet will be collected, please do not use the question booklet for answering.

ENTITY-RELATIONSHIP-MODEL (ERM)*

5 POINTS

Problem 1. Draw a diagram according to the Entity-Relationship-Model (ERM), which captures the following properties of and relationships among the five entity sets *Book*, *Author*, *Publisher*, *Store*, and *Comic*. Make sure that you include all the multiplicities, referential integrity constraints, and keys that are indicated in the following description into your ERM diagram.

1. A *Book* has a *title*, a *release date*, and a unique *ISBN* number.
2. A *Book* is written by one or more *Author*'s.
3. A *Book* is sold in zero or more *Store*'s.
4. A *Book* is released by exactly one *Publisher*, which must be a valid reference to this *Publisher*.
5. An *Author* may have a contract with zero or more *Publisher*'s.
6. *Store*, *Author* and *Publisher* entities each have a unique *name*.
7. A *Comic* is a special kind of a *Book*, which has the additional property *graphic novel*.

TRANSLATING AN ERM INTO A RELATIONAL DATABASE SCHEMA*

10 POINTS

Problem 2. Consider the following ERM with the ternary relationship *merchandise* as it is depicted in Figure 1.

- Briefly describe what this ERM expresses, and give an example for a set of entities with their corresponding relationships that conform to this ERM.
2 POINTS
- Translate the given ERM into an equivalent ERM that consists only of binary relationships.
2 POINTS
- Translate the ERM into a set of relation schemas of the form $R(A, B, \dots)$, where R denotes the relation name and A, B , etc. denote its attribute names. Underline all the key attributes of each relation.
(Note: you may either use the ERM you provided in b) or the ERM of Figure 1 for this translation.)
3 POINTS
- Implement the relation schema you provided in c) by providing a set of CREATE TABLE statements with proper key- and foreign-key constraints in SQL.
3 POINTS

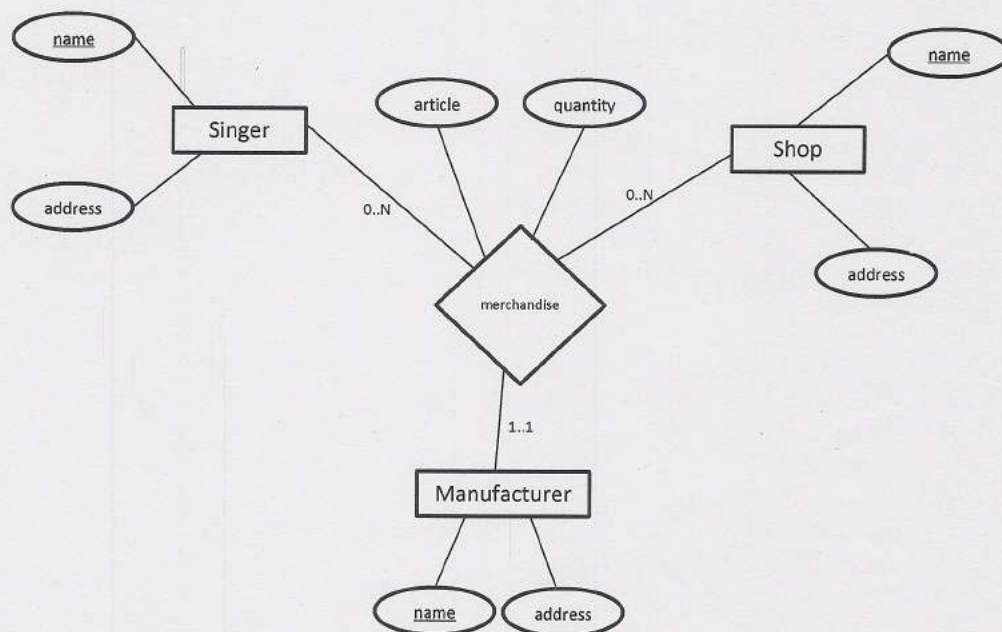


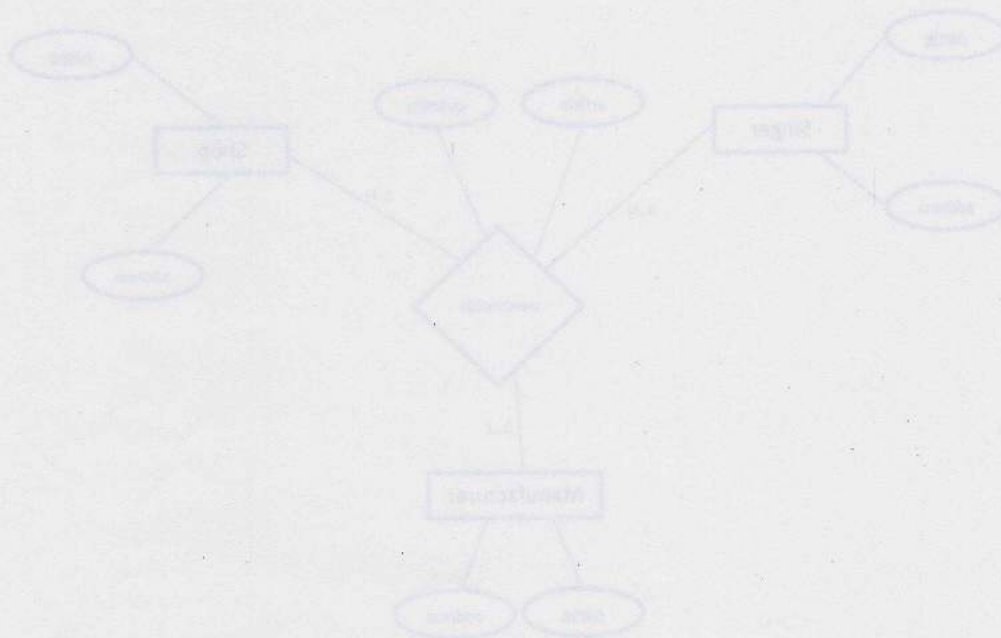
Figure 1: An ERM with a ternary relationship.

RELATIONAL DATA MODEL VS. SQL

10 POINTS

Problem 3. Briefly answer the following questions about the relational data model and SQL.

- Describe the basic properties of the relational data model.
2 POINTS
- Does every table in SQL need to have a primary key? Justify your answer.
2 POINTS
- Describe one of the three ways in which the semantics of SQL was defined in the lecture.
2 POINTS
- How is the relational data model different from SQL? Demonstrate the difference by specifying two join operations, one in relational algebra and one in SQL, that employ the same join condition but yield different results. Illustrate this by an example that takes the same data as input for your two join operations, and show the results for each of them.
4 POINTS



SQL QUERIES*

8 POINTS

Problem 4. Consider the example data provided in tables **Employees** and **Departments** below. You may assume that **name** is the *primary key* of **Departments**, and **d_name** is a *foreign key* of **Employees** that references the primary key of **Departments**. Note that an employee can be assigned to multiple departments, and a salary is specific to the work done in that department.

name	salary	d_name
John	\$10 000	R & D
John	\$8 000	ICT
Paul	\$12 000	HR
George	\$8 000	ICT
George	\$7 000	R & D
Susan	\$11 000	Fin.
Susan	\$7 000	ICT

Table 1: Employees

name	manager
R & D	Bill
HR	Bob
ICT	Bill
PR	Helen
Fin.	Alan
Law	Bob

Table 2: Departments

- Give all departments in which all employees have different salaries.
2 POINTS
- Give all pairs of employees that have the same salary in the same department. Make sure you give every pair just once.
2 POINTS
- Give all employees assigned to all departments in which Bill is manager.
2 POINTS
- Give all employees assigned to any department in which Bob is manager.
2 POINTS

FUNCTIONAL DEPENDENCIES*

10 POINTS

Problem 5.

- a) Which of the following two statements about functional dependencies are correct?
Briefly justify your answer.
- a.1) For every relation, there exist at least trivial functional dependencies.
2 POINTS
- a.2) If, for a relation, there exist only trivial functional dependencies, then the only key of that relation is the set of all its attributes.
2 POINTS
- b) Suppose we are given a relation schema $R(A, B, C, D)$ for which we know that the functional dependencies $C \rightarrow AD$, $A \rightarrow B$, and $D \rightarrow A$ hold.
- b.1) Determine all the keys that this relation has.
2 POINTS
- b.2) Determine all the superkeys that this relation has.
2 POINTS
- b.3) Apply Armstrong's axioms (but not the splitting and the combining rules) to determine all the completely non-trivial functional dependencies that you can derive from the above functional dependencies.
2 POINTS

SCHEMA NORMALIZATION*

8 POINTS

Problem 6. Consider the relation instance **Orders** as shown below in Table 3. Suppose we know that this relation instance has as its key $\{shop_id, client_id, item_id\}$ and the functional dependencies $client_id \rightarrow \{client_address, client_name\}$, $\{shop_id, client_id, item_id\} \rightarrow delivery_date$ and $item_id \rightarrow price$.

shop_id	client_id	item_id	price	delivery_date	client_address	client_name
1	1201	4	\$10	14/01/2013	17, Blank Avenue	Jane Doe
2	1444	17	\$23	18/01/2013	206, E-Street	Jack Daniels
2	2861	14	\$11	13/02/2013	48, Blue Square	Bob Rope
1	1444	12	\$60	16/01/2013	206, E-Street	Jack Daniels
1	2741	5	\$16	15/01/2013	28, High Road	Jack Russell

Table 3: Orders

- Briefly explain the problems that arise with this database schema.
2 POINTS
- Check whether this relation instance is in the third normal form (3NF). Justify your answer.
2 POINTS
- How would you normalize this relation schema, and what are the relation instances that you obtain for your normalized schema?
4 POINTS

PRIMARY VS. SECONDARY INDEX STRUCTURES

10 POINTS

Problem 7. Suppose we are given a relation schema $R(A, B)$, and we would like to organize the records that are to be stored for this relation in a sequential file with a fixed block size. Further assume that we would like to create two index structures to support an efficient access to these records by the values of the A and B attributes. We intend to do this by creating

- a primary index on the A attribute, and
- a secondary index on the B attribute.

Please answer the following questions.

a) How does the sequential file have to be sorted? Justify your answer.
2 POINTS

b) Which options are possible to handle duplicate search keys within such a sequential file?
3 POINTS

c) Which of the following index structures are possible as the primary index on attribute A and the secondary index on attribute B , respectively: a sparse or dense index file, a B^+ -tree, a linear-hashing table, or an extensible-hashing table.

(Note: copy Table 4 into your answer booklet and fill in your answers by marking the check boxes.)
5 POINTS

Index Structure	Primary Index on A	Secondary Index on B
Sparse Index File	☐	☐
Dense Index File	☐	☐
B^+ -Tree Index	☐	☐
Linear-Hashing Table	☐	☐
Extensible-Hashing Table	☐	☐

Table 4: Template for question c).

INDEXING VIA B⁺ TREES

8 POINTS

Problem 8. Suppose we are given the following set of numerical search keys:

{7, 1, 4, 10, 9, 3, 5, 12}

- a) Create a B⁺-tree index of order $n = 3$ that allows for efficient lookups of records associated with these keys.
5 POINTS
- b) Generally, there are two options for creating such a B⁺-tree index, when the set of all search keys (and pointers to records) is known in advance:
- Insert each search key one by one.
 - Bulkloading: first sort all the search keys, and then build the B⁺-tree by starting from the leaf nodes.

Which one would you prefer for solving a), and why?

3 POINTS

INDEXING VIA LINEAR HASHING

6 POINTS

Problem 9. Consider the linear-hashing index as it is depicted in Figure 2. Assume that the current configuration of the index has the following parameters: $n = 5$, $b = 2$ and $r = 8$.

000	00000
	11000
001	10101
	11101
010	10010
011	01111
100	00100
	11100

Figure 2: A linear-hashing index.

Insert the following sequence of binary search keys (one by one and in the given order) into this linear-hashing index.

(00111, 01000, 01001)

Show the complete configuration of the linear-hashing index structure and all its parameters after each insertion.

XML DATA MODEL*

10 POINTS

Problem 10. Consider the XML-Schema specification as it is depicted in Figure 3.

- a) Provide a smallest-possible XML document that is well-formed and valid with respect to this XML-Schema specification.
3 POINTS
- b) Briefly explain the kinds of constraints which are specified in this XML-Schema file, and which could also be specified in a document-type definition (DTD).
2 POINTS
- c) Change the XML-Schema file such that the name attribute becomes the key of the PROFESSOR element, and that the profID attribute of the COURSE element refers to it as a foreign key.
(Note: you do not have to copy the entire XML-Schema specification to answer this question. It is sufficient to just specify the necessary xs:key and xs:keyref constraints and to indicate into which xs:element they should be included.)
3 POINTS
- d) Give the names of all the element nodes that the following XPath expression may return, when this expression is evaluated against an XML document that is well-formed and valid with regard to the XML-Schema specification shown in Figure 3.

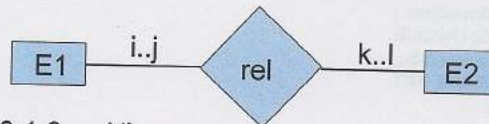
```
//COURSE//PROF[./@profID = "BobGates"]//following-sibling::*
```

2 POINTS

```
<?xml version="1.0"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="UNIVERSITY">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="PROFESSOR" minOccurs="1" maxOccurs="unbounded">
          <xs:complexType>
            <xs:attribute name="name" type="xs:string" use="required" />
          </xs:complexType>
        </xs:element>
        <xs:element name="BOOK" minOccurs="1" maxOccurs="unbounded">
          <xs:complexType>
            <xs:attribute name="name" type="xs:string" use="required" />
          </xs:complexType>
        </xs:element>
        <xs:element name="COURSE" minOccurs="1" maxOccurs="unbounded">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="PROF" minOccurs="1" maxOccurs="3">
                <xs:complexType>
                  <xs:attribute name="profID" type="xs:string" use="required" />
                </xs:complexType>
              </xs:element>
              <xs:element name="HANDBOOK" minOccurs="1" maxOccurs="4">
                <xs:complexType>
                  <xs:attribute name="bookID" type="xs:string" use="required" />
                </xs:complexType>
              </xs:element>
            </xs:sequence>
            <xs:attribute name="name" type="xs:string" use="required" />
          </xs:complexType>
        </xs:element>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
  <xs:key name="bookKey">
    <xs:selector xpath="./COURSE/HANDBOOK" />
    <xs:field xpath="@bookID" />
  </xs:key>
  <xs:keyref name="bookRef" refer="bookKey">
    <xs:selector xpath="./BOOK" />
    <xs:field xpath="@name" />
  </xs:keyref>
</xs:schema>
```

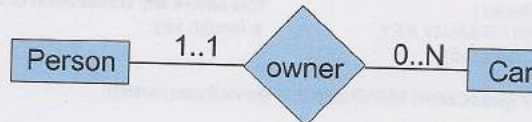
Figure 3: An XML-Schema file.

Multiplicities of Relationships in ERM



$i, j, k, l \in \{0, 1, 2, \dots, N\}$

- For each entity e_2 of E_2 there are at least i (at most j) entities of E_1 associated to e_2 ;
- For each entity e_1 of E_1 there are at least k (at most l) entities of E_2 associated to e_1 ;



More Ways to Connect Subqueries...

Assume s is a scalar value or an attribute,
 R is a unary relation whose tuples are of the same type.

EXISTS (R) iff relation R is not empty

s IN (R) iff s is in relation R , detects the membership in a bag

s NOT IN (R) iff s is not in relation R

s <> ALL (R) iff s <> all tuples in R ($=$)

s = ANY (R) iff s = a tuple in R ($<>$)

s > ALL (R) iff s is greater than all the values in the relation R ($<$, $<=$, $>=$, $=$, $<>$)

s > ANY (R) iff s is greater than at least one value in the relation R ($<$, $<=$, $>=$, $=$, $<>$)

NOT EXISTS (R), **NOT s > ALL (R)**, **NOT s > ANY (R)**

SELECT *

FROM R

WHERE (R.A, R.B) <> ALL

(SELECT S.A, S.B

FROM S);

(However NOT correct in Standard SQL;
 correct in Oracle.)

SELECT *

FROM R

WHERE R.A <> ALL

(SELECT S.A

FROM S

WHERE R.B = S.B);



Defining Foreign Keys

```
CREATE TABLE MovieExec (  
  name CHAR(30) UNIQUE,  
  address VARCHAR(255),  
  certN INT PRIMARY KEY,  
  netWorth INT);
```

There are two syntactic options for defining the foreign key of a relation in SQL:

```
CREATE TABLE Studio (  
  name CHAR(30) PRIMARY KEY,  
  address VARCHAR(255),  
  presCertN INT REFERENCES MovieExec(certN));
```

This is only possible if the foreign key contains exactly one attribute.

```
CREATE TABLE Studio (  
  name CHAR(30) PRIMARY KEY,  
  address VARCHAR(255),  
  presCertN INT,  
  FOREIGN KEY (presCertN) REFERENCES MovieExec(certN));
```

This allows any combination of attributes to be a foreign key.



A Complete Set of Inference Rules

Armstrong's Axioms:

The following three axioms allow for deriving any FD that follow from a given set of FD's.

1. Reflexivity:

If $\{B_1, B_2, \dots, B_m\} \subseteq \{A_1, A_2, \dots, A_n\}$,
then $A_1, A_2, \dots, A_n \rightarrow B_1, B_2, \dots, B_m$ ("trivial FDs")

2. Augmentation:

If $A_1, A_2, \dots, A_n \rightarrow B_1, B_2, \dots, B_m$,
then $A_1, A_2, \dots, A_n, C_1, C_2, \dots, C_k \rightarrow B_1, B_2, \dots, B_m, C_1, C_2, \dots, C_k$
for any set of attributes $C_1, C_2, \dots, C_k$ ("non-trivial FDs")

3. Transitivity:

If $A_1, A_2, \dots, A_n \rightarrow B_1, B_2, \dots, B_m$ and $B_1, B_2, \dots, B_m \rightarrow C_1, C_2, \dots, C_k$,
then $A_1, A_2, \dots, A_n \rightarrow C_1, C_2, \dots, C_k$ ("Closure algorithm")



Normal Forms Of Relations

In the following, let X be a set of attributes and Y be a single attribute of a relation R , and let F be a given set of FD's on R .

R is in **first normal form** (1NF), if all attribute values of every tuple are atomic. All the relations we consider are in 1NF.

R is in **second normal form** (2NF), if for every non-trivial FD $X \rightarrow Y$ in F it holds that X is not a proper subset of a key of R or Y is an attribute of a key of R .

R is in **third normal form** (3NF), if for every non-trivial FD $X \rightarrow Y$ in F it holds that X is a superkey of R or Y is an attribute of a key of R .

R is in **Boyce-Codd normal form** (BCNF), if for every non-trivial FD $X \rightarrow Y$ in F it holds that X is a superkey of R .



General Form Of A Location Step

axis :: node-test [predicate₁] ... [predicate_n] (n ≥ 0)

child::name[attribute::type="warning"][position()=5]

XPath v1.0 (and v2.0) knows 13 axes:

- the **child** axis contains the element and text children of the context node;
- the **descendant** axis contains the element and text descendants of the context node;
- the **parent** axis contains the parent of the context node;
- the **ancestor** axis contains the ancestors of the context node;
- if the context node is an element or text node the **following-sibling** axis contains all the following element or text siblings of the context node;
if the context node is an attribute node, the **following-sibling** axis is empty;
- if the context node is an element or text node the **preceding-sibling** axis contains all the preceding element or text siblings of the context node;
if the context node is an attribute node, the **preceding-sibling** axis is empty;



Normal Forms of Relations

In the following, let X be a set of attributes and Y be a single attribute of a relation R , and let F be a given set of FD's on R .

R is in first normal form (1NF), if all attribute values of every attribute are atomic. In the relations we consider, this is true.

R is in second normal form (2NF), if for every non-trivial FD $X \rightarrow Y$ in F , X is not a proper subset of a key of R , or Y is an attribute of a key of R .

R is in third normal form (3NF), if for every non-trivial FD $X \rightarrow Y$ in F , X is a superkey of R , or Y is an attribute of a key of R .

R is in Boyce-Codd normal form (BCNF), if for every non-trivial FD $X \rightarrow Y$ in F , X is a superkey of R .

General Form of a Location Step

node: node-list [predicate] ... [predicate] (n20)

where node-list is a list of nodes, and predicate is a predicate.

XPBth v1.0 (and v2.0) knows 13 axes:

- The first axis is the horizontal axis, and is denoted by the symbol x .
- The second axis is the vertical axis, and is denoted by the symbol y .
- The third axis is the depth axis, and is denoted by the symbol z .
- The fourth axis is the time axis, and is denoted by the symbol t .
- The fifth axis is the temperature axis, and is denoted by the symbol θ .
- The sixth axis is the pressure axis, and is denoted by the symbol p .
- The seventh axis is the density axis, and is denoted by the symbol ρ .
- The eighth axis is the viscosity axis, and is denoted by the symbol μ .
- The ninth axis is the surface tension axis, and is denoted by the symbol σ .
- The tenth axis is the electrical conductivity axis, and is denoted by the symbol κ .
- The eleventh axis is the thermal conductivity axis, and is denoted by the symbol λ .
- The twelfth axis is the mass diffusivity axis, and is denoted by the symbol D .
- The thirteenth axis is the mass diffusivity axis, and is denoted by the symbol D .